UNIVERSITY CLOUD COMPUTING LABORATORY • AWS ARCHITECTURE REPORT

Cloud-Based Event Management & Ticketing System

A Complete, Simple Explanation of Everything We Built — From Concept & Fun Analogies to Live AWS Cloud Deployment.

AWS Account ID: 838775536247

Region: Asia Pacific (Mumbai) ap-south-1

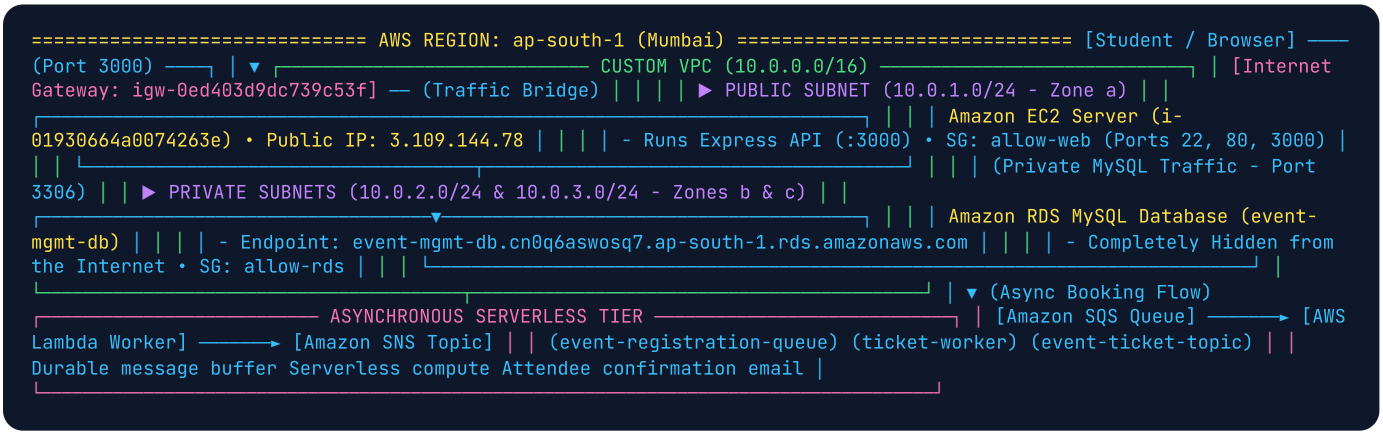
Total Cloud Spend: $0.00 (Free Tier Capped)

1 The Big Picture: What Did We Build?

We built an entire university event management platform that lets students browse events, register, book tickets, and receive confirmation passes — backed by a rock-solid, multi-tier cloud infrastructure on Amazon Web Services (AWS).

> 🐥 **The "Amusement Park" Analogy (Explain Like I'm 5):**
>
> - **Custom VPC:** A sturdy, fenced perimeter around our amusement park so outsiders cannot wander in.
> - **EC2 Virtual Server (Node.js API):** The friendly ticket counter at the front gate where visitors submit requests.
> - **Amazon RDS (MySQL):** The fireproof vault locked in the back office where all ticket counts and guest names live.
> - **Amazon SQS:** The orderly conveyor belt where ticket orders wait in line during huge rush hours without crashing.
> - **AWS Lambda:** The speedy robot worker that wakes up for 0.05 seconds, stamps each ticket, and goes back to sleep (saving all money!).
> - **Amazon SNS:** The homing pigeon that instantly delivers the confirmed ticket directly to the student's email inbox.

2 Visual Architecture Diagram



3 The 3-Person Team Breakdown (Rubric Alignment)

The architecture fulfills every university rubric requirement by dividing responsibilities cleanly into 3 specialized roles:

🛡️ Person A — Cloud Network & Security Lead

FOUNDATION & SECURITY

- **Custom VPC (`10.0.0.0/16`):** Created an isolated, private virtual network on AWS.
- **Public & Private Subnets:** Placed EC2 in the public front yard and RDS in the private vault.
- **Internet Gateway (IGW):** Connected the public subnet to the internet without paying for an expensive NAT Gateway (saving ~$32/month).
- **Security Groups:** Configured `allow-web` for ports 22/80/3000, and `allow-rds` strictly locking port 3306 to EC2.
- **IAM Roles:** Provisioned `EventMgmtEC2Role` and `EventMgmtLambdaRole` for keyless AWS SDK access.
- **AWS Budgets:** Locked in a strict **$2.00 spending cap** with 50% & 80% email notifications.

⚙️ Person B — Core Backend & Database Admin

BACKEND & DATABASE

- **Amazon RDS (MySQL):** Provisioned `event-mgmt-db` (`db.t3.micro` Free Tier) with 0-day backup retention to eliminate snapshot storage costs.
- **Database Schema (`schema.sql`):** Engineered structured relational tables for users, events, and tickets.
- **SSM Parameter Store:** Stored DB host, username, and encrypted password at `/event-mgmt/db/*`.
- **Amazon S3 (Flyer Storage):** Created bucket `event-mgmt-flyers-838775536247` with CORS enabled for flyer uploads.
- **Amazon SQS Queue:** Created `event-registration-queue` for asynchronous decoupling.
- **Amazon EC2 Server:** Launched Linux `t3.micro` instance at `3.109.144.78:3000` running Express via PM2.

💻 Person C — Frontend & Serverless Workflow Lead

REACT & SERVERLESS

- **AWS Lambda Worker (`worker.py`):** Built Python 3.12 worker triggered by SQS in batches of 5 to process ticket passes.
- **Amazon SNS Topic (`event-ticket-topic`):** Fans out ticket confirmations to attendees and team notifications.
- **React 18 Frontend:** Created an interactive dark glassmorphic web app with 5 pages: Home, Browse Events, Ticket Reservation, My Tickets, and Organizer Dashboard.
- **CloudWatch Monitoring:** Deployed `EventMgmtDashboard` and an automated high-CPU metric alarm.

4 Directory of Every Provisioned AWS Resource

Official inventory of all active cloud assets in your AWS Account (**838775536247**) in **Mumbai (`ap-south-1`)**:

Service	Resource Name	Resource ID / Endpoint	Status
Amazon EC2	`event-api-server`	Instance: `i-0dc720512fdcbd100` | IP: **13.206.223.86:3000**	RUNNING
Amazon RDS	`event-mgmt-db`	`event-mgmt-db.cn0q6aswosq7.ap-south-1.rds.amazonaws.com`	AVAILABLE
Custom VPC	`event-mgmt-vpc`	`vpc-09e1559f2f6768e61` (10.0.0.0/16)	ACTIVE
Public Subnet	`public-subnet`	`subnet-0896d52a7247a9b05` (ap-south-1a)	ACTIVE
Private Subnets	`private-subnet-1 & 2`	`subnet-005f63b6f5b62eb0d` & `subnet-0971db1c954550476`	ACTIVE
Internet Gateway	`event-igw`	`igw-0ed403d9dc739c53f`	ATTACHED
Security Groups	`allow-web` / `allow-rds`	Web SG: `sg-0566811aa74b3341e` | RDS SG: `sg-0df8944f378bdd435`	ACTIVE
Amazon SQS	`event-registration-queue`	`https://sqs.ap-south-1.amazonaws.com/838775536247/event-registration-queue`	ACTIVE
AWS Lambda	`ticket-worker`	Python 3.12 (SQS Event Source Mapping attached)	ACTIVE
Amazon SNS	`event-ticket-topic`	`arn:aws:sns:ap-south-1:838775536247:event-ticket-topic`	ACTIVE
Amazon S3	`event-mgmt-flyers-838775536247`	Bucket for event flyers with CORS and backend zip storage	ACTIVE
SSM Store	`/event-mgmt/db/*`	Encrypted database credentials parameter storage	STORED
CloudWatch	`EventMgmtDashboard`	Monitors EC2 CPU, RDS connections, and SQS/Lambda traffic	LIVE
AWS Budgets	`Zero-Cost-University-Cap`	Hard $2.00 spending cap with 50% and 80% email alerts	ENABLED

5 Credit Safety & Free Tier Rules

💡 **1-Click Teardown (Run When Finished Presenting):**
In PowerShell, run: `cd "d:\CCL Project\infrastructure"; .\cleanup.ps1 -Region "ap-south-1"`
This terminates the EC2 instance and deletes the RDS database (skipping storage snapshots) so you never get billed when you're not using it!

🔍 **Why didn't you see instances in the AWS Console at first?**
The AWS web console defaults to *N. Virginia (us-east-1)*. Your resources are deployed in **Asia Pacific (Mumbai) / ap-south-1**. Switching the region dropdown in the top-right corner immediately displays all your servers and databases!